



Java

Clase 08 | Manejo de Excepciones y Módulos

Índice

Clase 08

Clase 8 | Manejo de Excepciones y Módulos

- Manejo de excepciones en Java.
- Uso de try, catch, finally.
- Creación de excepciones personalizadas.
- Organización del código con paquetes y módulos.
- Importación y uso de clases externas.

Clase 09

Clase 9 | Introducción a Git y Control de Versiones

- Conceptos fundamentales de control de versiones.
- Instalación y configuración de Git.
- Flujo de trabajo básico con Git:
 - Creación de repositorios.
 - Comandos básicos: add, commit, push.
- Colaboración en proyectos mediante GitHub.
- Buenas prácticas de versionado.

Objetivos de la Clase



Excepciones en Java

Comprender tipos y manejo de excepciones.



Organización del Código

Aprender sobre paquetes y módulos.



Aplicación Práctica

Resolver problemas reales en TechLab.

Excepciones

Introducción a las Excepciones en Java

Las excepciones en Java son eventos inesperados que interrumpen el flujo normal de ejecución de un programa.

Permiten manejar errores y condiciones anómalas de forma controlada, evitando que el programa se bloquee.



Uso de try, catch, finally

try

Envuelve el código que podría generar una excepción. Aquí colocamos operaciones arriesgadas como lectura de archivos, conexiones a bases de datos o cálculos que podrían fallar.

catch

Captura y maneja la excepción si ocurre. Podemos tener múltiples bloques catch para diferentes tipos de excepciones, permitiendo responder de manera específica a cada tipo de error.

finally

Se ejecuta siempre, independientemente de si hubo excepción o no. Es crucial para cerrar recursos (archivos, conexiones, etc.) y asegurar la limpieza adecuada del programa, evitando fugas de memoria.



Tipos de Excepciones en Java

Checked Exceptions

Heredan de la clase `Exception` y requieren manejo explícito en el código mediante `try-catch` o `throws`. El compilador verifica que estas excepciones sean manejadas.

- `IOException`: Para operaciones de entrada/salida
- `SQLException`: Para errores en bases de datos
- `FileNotFoundException`: Cuando no se encuentra un archivo

```
try {  
    FileReader file = new  
        FileReader("archivo.txt");  
} catch (FileNotFoundException e) {  
    System.out.println("Archivo no  
        encontrado");  
}
```

Tipos de Excepciones en Java

Unchecked Exceptions

Heredan de `RuntimeException` y no requieren declaración explícita. Representan errores de programación que normalmente pueden prevenirse.

- `NullPointerException`: Al acceder a un objeto null
- `ArrayIndexOutOfBoundsException`: Índice fuera de rango
- `ArithmeticException`: División por cero

```
int[] array = new int[3];
try {
    System.out.println(array[5]); //
Error!
} catch (ArrayIndexOutOfBoundsException e)
{
    System.out.println("Índice inválido");
}
```




Creación de Excepciones Personalizadas

Las excepciones personalizadas nos permiten crear nuestros propios tipos de excepciones específicas para nuestra aplicación. Se crean extendiendo la clase `Exception` o alguna de sus subclases, lo que nos permite definir errores específicos del dominio de nuestra aplicación.

Para crear una excepción personalizada, simplemente creamos una nueva clase que herede de `Exception` y definimos los constructores necesarios. Esto nos permite lanzar y capturar excepciones que tienen sentido en el contexto de nuestro programa.

Ejemplo de Excepcion Personalizada

Por ejemplo, podemos crear una excepción personalizada llamada **ProductoNoEncontradoException** para representar un error cuando se intente acceder a un producto que no existe en nuestro sistema:

```
public class ProductoNoEncontradoException extends Exception {  
    public ProductoNoEncontradoException(String mensaje) {  
        super(mensaje);  
    }  
}
```

Ejemplo de cómo lanzar y capturar esta excepción:

```
public class Inventario {  
    public Producto buscarProducto(String codigo) throws ProductoNoEncontradoException {  
        Producto producto = baseDeDatos.buscar(codigo);  
        if (producto == null) {  
            throw new ProductoNoEncontradoException(  
                "No se encontró el producto con código: " + codigo  
            );  
        }  
        return producto;  
    }  
}  
  
try {  
    Producto producto = inventario.buscarProducto("ABC123");  
} catch (ProductoNoEncontradoException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

Problema: Validar cantidad de café

En la tienda de café Sibeliuss necesitan validar que la cantidad de gramos de café solicitada por el cliente esté dentro de los límites permitidos. La máquina solo puede procesar entre 5 y 30 gramos por taza, y necesitamos lanzar una excepción personalizada cuando la cantidad esté fuera de este rango.

Para solucionar esta necesidad, necesitamos:

- Crear una excepción personalizada CantidadInvalidaException
- Validar que la cantidad solicitada esté entre 5 y 30 gramos
- Lanzar la excepción con un mensaje claro cuando la cantidad esté fuera de rango

Este sistema asegura que cada pedido de café cumpla con las especificaciones técnicas de la máquina y evita desperdicios o preparaciones inadecuadas.

► [Hacé clic acá para ver la solución](#)

Este sistema asegura que cada pedido de café cumpla con las especificaciones técnicas de la máquina y evita desperdicios o preparaciones inadecuadas.

▼ [Hacé clic acá para ver la solución](#)

```
public class CantidadInvalidaException extends Exception {
    public CantidadInvalidaException(String mensaje) {
        super(mensaje);
    }
}

public void prepararCafe(int gramos) throws CantidadInvalidaException {
    if (gramos < 5 || gramos > 30) {
        throw new CantidadInvalidaException(
            "La cantidad de " + gramos + " gramos está fuera del rango permitido (5-30g)"
        );
    }
    System.out.println("Preparando café con " + gramos + " gramos");
}
```

Paquetes

Organización del Código con Paquetes

Los paquetes en Java son una forma fundamental de organizar y estructurar nuestro código, actuando como contenedores para agrupar clases relacionadas. Esta organización no solo mejora la legibilidad del código, sino que también ayuda a evitar conflictos de nombres entre clases.

Al utilizar paquetes, podemos:

- Organizar las clases de manera jerárquica y lógica
- Controlar el acceso y la visibilidad de las clases
- Facilitar la reutilización y el mantenimiento del código

La convención de nomenclatura estándar utiliza el formato **com.empresa.proyecto.modulo**, lo que asegura nombres únicos y una estructura clara del proyecto.



Problema: Dividir en Paquetes Lógicos

Los paquetes en Java nos ayudan a organizar nuestro código de manera ordenada, similar a cómo organizamos archivos en carpetas en nuestra computadora. 📁

Estructura Básica

La estructura típica de paquetes sigue este patrón:

- com.empresa.proyecto.modelo
- com.empresa.proyecto.servicios
- com.empresa.proyecto.util

¿Qué va en cada paquete?

- modelo: Clases que representan datos (Usuario, Producto)
- servicios: Lógica del negocio
- util: Herramientas y funciones auxiliares
- dao: Acceso a datos

Para usar una clase de otro paquete, usamos la palabra clave **import**. Por ejemplo:

```
import com.empresa.modelo.Usuario;
```


Importación y Organización de Paquetes

Estructura

```
com.empresa.proyecto
├── modelo
│   ├── Usuario.java
│   └── Producto.java
├── servicios
│   ├── UsuarioServicio.java
│   └── ProductoServicio.java
└── util
    └── Validador.java
```

La organización en paquetes es fundamental para proyectos escalables. Se recomienda agrupar clases relacionadas funcionalmente y seguir convenciones de nomenclatura como "**com.empresa.proyecto**". Los paquetes más comunes incluyen: modelo (entidades), servicios (lógica de negocio), util (herramientas), dao (acceso a datos) y excepciones.

Ejemplo de Importación

```
import com.empresa.modelo.*;
import
com.empresa.servicios.UsuarioServicio;
import com.empresa.util.Validador;
```

Problema: Organización de Paquetes para Cafetería Sibeliuss

La cafetería Sibeliuss necesita organizar su sistema de manera eficiente y escalable. Para esto, necesitamos estructurar los diferentes componentes del negocio (productos, ventas, inventario, empleados) en paquetes lógicos que faciliten el mantenimiento y la expansión del sistema.

La estructura de paquetes debe considerar:

- Productos de café y repostería
- Gestión de ventas y recibos
- Control de inventario
- Administración de empleados

Esta organización permitirá mantener el código ordenado y facilitar la implementación de nuevas funcionalidades a medida que la cafetería crezca.

► [Hacé clic acá para ver la solución](#)

Esta organización permitirá mantener el código ordenado y facilitar la implementación de nuevas funcionalidades a medida que la cafetería crezca.

▼ [Hacé clic acá para ver la solución](#)

```
com.sibelius.cafeteria
├── modelo
│   ├── Producto.java
│   ├── Empleado.java
│   └── Venta.java
├── servicios
│   ├── VentaServicio.java
│   ├── InventarioServicio.java
│   └── EmpleadoServicio.java
├── util
│   ├── ReciboGenerador.java
│   └── CalculadorDescuento.java
└── excepciones
    ├── InventarioException.java
    └── VentaException.java
```

Recursos Adicionales



Documentación oficial

Consulta la documentación de Java para profundizar en los conceptos.



docs.oracle.com



Exception (Java Platform SE 8)

The class Exception and any subclasses that are not also subclasses of RuntimeException are checked...



Guía de métodos

Revisa la guía de Oracle sobre métodos en Java.

<https://docs.oracle.com/javase/specs/jls/se13/html/jls-11.html>



Videos recomendados

Mira "Organización del Código con Métodos" en YouTube.

https://youtube.com/playlist?list=PL954bYq0HsCXlwPGM2k2nxRVztH0wpPqc&si=GfIFagrxE_BjMaw4



Preguntas para Reflexionar



Robustez del Software

¿Cómo mejora el manejo de excepciones la calidad?



Excepciones Personalizadas

¿Cuándo usarlas en lugar de las predefinidas?



Organización del Código

¿Cómo facilita la evolución del proyecto?



Ejercicios prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, ha notado que algunas operaciones del backend podrían fallar. Por ejemplo, si un cliente busca un producto que no existe, el sistema podría simplemente fallar o retornar datos incoherentes. Matías y Sabrina desean manejar estas situaciones informando claramente al usuario o registrando el error para su posterior análisis.

Además, la cantidad de clases y paquetes crece, y el equipo ve la necesidad de mantener una estructura ordenada. Esto implica agrupar clases en paquetes según su responsabilidad (productos, clientes, pedidos, utilidades) e incluso considerar la modularización para controlar qué partes del código se exponen y a quién.

Ejercicio Práctico

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Manejo de Excepciones

- Crea el método `buscarProductoPorNombre(String nombre)` que lance `ProductoNoEncontradoException` si no halla el producto.
- Maneja la excepción en un main con try-catch.

Excepciones personalizadas

- Crea una `StockInsuficienteException` que se lance cuando se intente vender más unidades de las que hay en stock.
- Manejá la excepción en el código y mostrale al usuario un mensaje adecuado.

Ejercicio Práctico

Optativo | No entregable

Organización de paquetes

- Mueve las clases Producto, Bebida, Comida al paquete `com.techlab.productos`.
- Mueve las excepciones a `com.techlab.excepciones`.

Ajusta los imports en el main.

Módulos

- Crea un módulo `com.techlab.negocio`
- Exporta el paquete `com.techlab.productos`.

Simula que otro módulo `com.techlab.ui` importa y utiliza Producto.

Pre-Entrega de Proyecto

Revisión de Progreso

Obligatorio | Entregable

Revisión de progreso

Estamos acercándonos a un **punto clave en el desarrollo del proyecto**, donde tendrán la oportunidad de mostrar lo que han aprendido hasta ahora. Es un excelente momento para **compartir su progreso y recibir comentarios** que les ayudarán a pulir detalles y afinar sus ideas antes de la entrega final.

Aprovechen esta instancia para obtener una visión más clara de su trabajo y hacer los ajustes necesarios para llegar a un resultado óptimo.

Entregable

Como parte de tu avance en el curso y para prepararte para el desarrollo del **Trabajo Final Integrador (TFI)**, se te solicita que diseñes un sistema básico que permita gestionar información inicial sobre los productos de la empresa. Este proyecto servirá como la base sobre la cual construirás funcionalidades más complejas en las próximas semanas. Así, **podrán recibir comentarios que les permitirán fortalecer su proyecto y enfocarse en los detalles clave para la etapa final**.

Revisión de Progreso

Obligatorio | Entregable

Revisión de progreso

Esta instancia evaluativa es de carácter obligatorio y es un punto clave dentro de la cursada ya que nos permitirá evaluar tu progreso en el recorrido y asegurar que estás en el camino correcto en la construcción del **Proyecto Final Integrador**.

Este espacio de entrega está conformado por:



Cuestionario de Autoevaluacion



Pre-Entrega de Proyecto

Pre-Entrega de Proyecto

Obligatorio | Entregable

A partir de la **Clase N° 8** tendrás **7 días** de corrido para realizar la autoevaluación y la entrega en el campus virtual

Cuestionario de Autoevaluación

- Te permitirá reflexionar sobre tu propio aprendizaje, progreso y cumplimiento de las consignas o rúbricas previamente establecidas y en caso de ser necesario realizar las modificaciones o ajustes correspondientes antes de realizar la Pre-Entrega.
- Se encontrará disponible en la Ruta N°2 de Campus Virtual.

Pre-Entrega de Proyecto:

- Se evaluará la aplicación de los conocimientos adquiridos en la realización de un proyecto.
- La realización progresiva de los "**Ejercicios prácticos**" los guiará paso a paso hacia la realización de la "**Pre - Entrega**" y el "**Proyecto Integrador Final**"

Ejercicio Práctico

Obligatorio | Entregable

Vas a desarrollar un programa en Java que cumpla con las siguientes características:

Objetivo General

- Diseñar una aplicación en Java que permita **registrar, mostrar y gestionar productos**, así como **crear pedidos** que involucren varios productos. El sistema deberá hacer uso de **variables, operadores, colecciones (listas), POO (clases, objetos, encapsulamiento, herencia, polimorfismo), excepciones y organización de código en paquetes/módulos**.

Ejercicio Práctico

Obligatorio | Entregable

Requerimientos:

Ingreso de Productos

- El sistema debe permitir **agregar productos** con la siguiente información mínima:
 - **Nombre** del producto (cadena de texto, por ejemplo "Café Premium").
 - **Precio** (double, puede tener decimales).
 - **Cantidad en Stock** (int).
- Estos productos deben almacenarse en una **colección** (por ejemplo, `ArrayList<Producto>`).

Visualización de Productos

- Debe haber una **funcionalidad** que liste en pantalla todos los productos registrados, mostrando su **ID (o posición)**, **Nombre**, **Precio** y **Cantidad en Stock**.
- El ID puede ser autogenerado (por ejemplo, un contador estático) o la posición en la lista.

Ejercicio Práctico

Obligatorio | Entregable

Búsqueda y Actualización de Productos

- El sistema permitirá **buscar un producto** por su nombre o ID.
- Si se encuentra el producto, se mostrará su información completa.
- Opcionalmente, se podrá **actualizar** alguno de sus datos (precio o stock), validando que los valores sean coherentes (por ejemplo, que el stock no sea negativo).

Eliminación de Productos

- El sistema debe permitir **eliminar** un producto de la lista, identificándolo por su ID o posición en la colección.
- Antes de eliminar, el sistema podría pedir confirmación (opcionales).

Ejercicio Práctico

Obligatorio | Entregable

Creación de Pedidos

- Además de manejar productos, se sugiere agregar la **clase Pedido** (o **Orden**) que contenga:
 - Una **lista de productos** asociados.
 - Cantidad deseada de cada producto (por ejemplo, usando un objeto intermedio **LineaPedido** o similar).
- El sistema debe permitir **crear** un pedido nuevo:
 - a. Solicitar al usuario qué productos desea y en qué cantidad (validar que haya suficiente stock).
 - b. Calcular el **costo total** (sumando **precio * cantidad** de cada producto).
 - c. **Disminuir el stock** de cada producto si el pedido se confirma.
- Debe haber una **funcionalidad** para **mostrar** los pedidos realizados y su costo total, así como la lista de productos asociados.

Ejercicio Práctico

Obligatorio | Entregable

Menú Principal Interactivo

El programa presentará un **menú** con opciones, por ejemplo:

- **Agregar producto**
- **Listar productos**
- **Buscar/Actualizar producto**
- **Eliminar producto**
- **Crear un pedido**
- **Listar pedidos** (opcionales)
- **Salir**

El menú se repetirá hasta que se elija la opción de **Salir**.

Ejemplo del menú de opciones:



SISTEMA DE GESTIÓN - TECHLAB

- 1) Agregar producto
- 2) Listar productos
- 3) Buscar/Actualizar producto
- 4) Eliminar producto
- 5) Crear un pedido
- 6) Listar pedidos
- 7) Salir

Elija una opción: